COMP5339 Quiz 2 完整速答图

W7-W10 每节规则 + 71 道网站自测正确答案 + 代码/陷阱

W7.01 为什么数据库需要“时间能力”

题干: temporal DBMS / history / as-of / when known
结论: 时间数据库同时保存事实及其时间语义，因此能回答现在、过去和知识变更问题。
规则: 普通快照表只保留当前状态；UPDATE 会覆盖旧值。；Temporal DBMS 内建时间存储、查询和更新支持。；典型题：某事实何时在现实有效？数据库何时知道？过去某时刻是什么状态？
网站题答案: Q1=内建按时间存储、查询和更新的支持
□ 给普通表加一个 timestamp 不等于完整的 temporal support。

W7.02 Instant、Interval 与 Period

题干: instant / interval / period / overlap
结论: Instant 是时刻；interval 是时长；period 是有起止边界的时间段。
规则: 半开区间 [start,end) 最安全：start <= t AND t < end。；相邻期间 [a,b) 与 [b,c) 不重叠，也不会重复计数。；duration = end - start；period 同时记录两个边界。
网站题答案: Q1=不包含

```
/ : WHERE ts >= :start AND ts < :end
```

□ BETWEEN 两端都包含，连续窗口容易在边界重复。

W7.03 User-defined、Valid 与 Transaction Time

题干: valid time / transaction time / bitemporal
结论: Valid time 问现实何时为真；transaction time 问数据库何时保存/知道；两者都有就是 bitemporal。
规则: User-defined time 只是普通 DATE/TIMESTAMP，DBMS 不理解其语义。；Valid time 可记录迟到更正；transaction time 通常由系统维护且历史不可随意改写。；同一事实可在现实中早已有效，但之后才录入数据库。
网站题答案: Q1=Transaction time
□ 录入时间不是事实发生时间；不要把 valid 与 transaction time 混用。

W7.04 Temporal Tables、NOW 与查询语义

题干: current / sequenced / nonsequenced / NOW
结论: Current 查当前版本；sequenced 对每个时间点按时间语义运算；nonsequenced 把时间列当普通列。
规则: 当前行常用 end='infinity' 或系统最大时间表示仍有效。；历史更新通常关闭旧 period，再插入新版本。；Sequenced join 除普通 key 外还必须令有效期间重叠。
网站题答案: Q1=Bitemporal table

```
/ : overlap: a.start < b.end AND b.start < a.end
```

□ NOW 是移动概念；把它写死成一次执行时间会失去持续有效语义。

W7.05 传统 SQL、SQL:2011 与 PostgreSQL Range

题干: SQL:2011 PERIOD / PostgreSQL range / GiST
结论: 原生 period/range 类型让数据库理解重叠与包含，并可用空间式索引加速。
规则: SQL:2011 提供 PERIOD 和 system-versioned temporal table 概念。；PostgreSQL tsrange/tstzrange 可表达 [start,end)，&& 表示重叠，@> 表示包含。；B-tree 适合标量顺序；range overlap 常用 GiST。
网站题答案: Q1=把开始列与结束列在元数据中组合；Q2=重叠 overlaps

```
/ : valid_period && tstzrange(:s,:e,'[)')
CREATE INDEX ... USING gist(valid_period);
```

□ 分别给 start、end 建普通索引并不等价于 range overlap 索引。

W7.06 Point-based 与 Sequence-based 时间序列

题干: point-based / sequence-based time series
结论: Point-based 每个时间点一行；sequence-based 一行保存有序的时间和值序列。
规则: 点式结构易过滤、连接和更新；序列式适合整段读取与压缩。；序列式必须保证 timestamp 与 value 一一对应且顺序稳定。；若会频繁按时间范围筛选或 join，点式通常更直接。
网站题答案: Q1=Point-based
□ 多个平行数组很容易长度或排序不一致，造成时间和值错配。

W7.07 时间过滤与聚合 SQL

题干: WHERE / GROUP BY / HAVING / aggregate
结论: 先 WHERE 筛行，再 GROUP BY 分组，再算聚合，最后 HAVING 筛组。
规则: WHERE 不能直接筛聚合结果；HAVING 可以。；SELECT 中非聚合列必须出现在 GROUP BY。；MIN/MAX/AVG/COUNT 把每组多行压成一行。
网站题答案: Q1=WHERE；Q2=HAVING AVG(amount) > 5；Q3=按 shop 和 month

```
/ : SELECT k, AVG(v) avg_v
FROM t WHERE ts>=:s AND ts<:e
GROUP BY k HAVING COUNT(*)>=2
ORDER BY k;
```

□ SQL 书写顺序不是逻辑执行顺序；HAVING 不是 WHERE 的替代品。

W7.08 CTE：把复杂 SQL 拆成可检查步骤

题干: WITH / CTE / readable SQL
结论: CTE 给中间查询命名，适合把过滤、转换、聚合拆成可检查步骤。
规则: 后续 CTE 可引用前面的 CTE。；CTE 提高可读性与调试性，但不保证自动缓存或更快。；递归 CTE 需 WITH RECURSIVE；普通 CTE 不是循环。
网站题答案: Q1=用于命名中间查询步骤

```
/ : WITH clean AS (...), agg AS (SELECT ... FROM clean GROUP BY ...) SELECT * FROM agg;
```

□ 不要因为用了 WITH 就假定只执行一次或已物化。

W7.09 PostgreSQL 数组

题干: array_agg / unnest / PostgreSQL array
结论: array_agg 把多行聚成数组；unnest 把数组展开为多行。
规则: 必须在 array_agg 内写 ORDER BY 才能保证元素顺序。；PostgreSQL 数组通常从 1 开始索引。；多个数组分别聚合可能错位；可聚合 ROW/STRUCT 保持字段绑定。
网站题答案: Q1=a[1]

```
/ : array_agg(value ORDER BY ts)
SELECT unnest(values);
```

□ 外层 ORDER BY 不能保证 array_agg 内部顺序。

W7.10 Spark SQL 数组与索引陷阱

题干: collect_list / sort_array / struct / explode / element_at
结论: Spark 中先收集带 timestamp 的 struct，再排序并拆字段，才能保持多列对齐。
规则: collect_list 本身不保证顺序。；sort_array(collect_list(named_struct(...))) 先把整条记录一起排序。；explode 将数组元素变回多行；Spark [] 索引从 0 开始。
网站题答案: Q1=不保证

```
/ : sort_array(collect_list(named_struct('ts',ts,'v',v)))
explode(arr)
element_at(arr,1) -- SQL      1
```

□ Spark 的 bracket index 与 element_at 起点不同；不要混记。

W7.11 插入、更新与时间单位

题干: INSERT / overwrite / append / Unix seconds
结论: 先确认写入模式和时间单位；秒与毫秒错 1000 倍会生成荒谬日期。
规则: 显式列名 INSERT 更安全，避免 schema 顺序变化。；append 增加数据；overwrite/UPDATE 会替换，可能破坏历史。；10 位 Unix 常为秒，13 位常为毫秒；转换前检查数量级。
网站题答案: Q1=毫秒

```
/ : to_timestamp(epoch_seconds)
from_unixtime(epoch_ms/1000)
```

□ 不要看到整数就直接当秒；先检查日期范围。

W7.12 Tutorial：Bitcoin 时间序列完整流程

题干: Bitcoin tutorial / end-to-end temporal workflow
结论: 正确流程是读入→验证时间单位→转换/排序→过滤聚合→检查→可视化。
规则: 先看 schema、样例、NULL、重复和时间范围。；SQL 与图表必须使用相同过滤边界和聚合粒度。；图上的异常先回查原始记录，不能仅凭形状删数据。
网站题答案: Q1=字段名难以按日期范围查询索引；Q2=先验证查询结果的粒度、类型和行数
□ 漂亮图表不等于正确结果；时间单位和排序错误会产生可信但错误的图。

W8.01 什么是非结构化数据

题干: structured / semi-structured / unstructured
结论: 结构化有固定表模式；半结构化有标签/键但结构可变；非结构化不适合直接放入固定行列。
规则: JSON/XML 通常是 semi-structured；文本、图像、音频通常是 unstructured。；非结构化不等于没有信息，需要特征提取后才能分析。；metadata 可把非结构化对象连接到结构化查询。
网站题答案: Q1=半结构化
□ 不要把 JSON 因为是文本文件就归为纯非结构化。

W8.02 监督学习、无监督学习与 ML Pipeline

题干: supervised / unsupervised / classification / regression
结论: 有 label 用 supervised；无 label 找结构用 unsupervised。离散标签=classification，连续数值=regression。
规则: Clustering 和 topic modelling 是常见无监督任务。；标准流程：split→只在 train 上 fit 预处理/模型→validation 调参→test 最终评估。；pipeline 保证训练与预测采用同一转换。
网站题答案: Q1=Clustering
□ 在全数据上先标准化、选特征或建词表会 data leakage。

W8.03 图像怎样存成数字

题干: pixel / channel / bit depth / raster / vector
结论: Raster 是像素网格；vector 是几何对象。图像数组常为 H×W×C。
规则: RGB 3 channels；grayscale 1 channel；binary mask 常为 0/1。；bit depth 决定可表示强度级数，如 8-bit 为 0–255。；模型输入常需 resize、normalize，并保持通道顺序一致。
网站题答案: Q1=Binary mask；Q2=0 或 1
□ 宽高/通道顺序、RGB/BGR、0–255/0–1 混淆会静默地产生错误。

W8.04 文本预处理

题干: tokenisation / stopwords / stemming / lemmatisation
结论: 文本预处理把原文变成稳定 token，但每一步都可能丢失有用信号。
规则: tokenise 分词；normalise 统一大小写/形式；stopword removal 删除高频功能词。；stemming 粗暴截词；lemmatisation 按语言规则还原词元。；任务相关：情感分析中 not 可能非常重要，不能机械删除。
网站题答案: Q1=还原规范词形并考虑语言规则
□ 更多清洗不一定更好；删除标点、大小写或停用词前要看任务。

W8.05 Count、Bag of Words、TF-IDF 与稀疏矩阵

题干: Count / Bag of Words / TF-IDF / sparse
结论: BoW 用词频向量；TF-IDF 降低到处出现的词权重，提高对少数文档有区分力的词。
规则: TF(t,d)=词在文档中的频率；IDF≈log(N/df(t))。；词汇表大而每篇只出现少量词，所以矩阵高度 sparse。；BoW 忽略词序；n-gram 可保留短局部顺序但维度更高。
网站题答案: Q1=1；Q2=该文档常见但其他文档少见的词

```
/ : tfidf(t,d)=tf(t,d) × log(N/df(t))
```

□ TF-IDF 高表示区分性强，不等于该词在整个语料最常见。

W8.06 文档相似度与向量距离

题干: cosine / Jaccard / Euclidean / document similarity
结论: 文本稀疏向量常用 cosine；集合重叠用 Jaccard；同尺度连续特征才常用 Euclidean。
规则: cosine 比较方向，较不受文档长度影响；1 最相似，0 正交。；Jaccard =|A∩B|/|A∪B|，忽略出现次数。；距离越小越近；相似度越大越近，题目先看指标方向。
网站题答案: Q1=1

```
/ : cos(A,B)=A·B/(||A|| ||B||)
```

□ 不要把 cosine similarity 与 cosine distance 的大小方向写反。

W8.07 Word2Vec、BERT 与 GPT

题干: Word2Vec / BERT / GPT / embedding
结论: Word2Vec 给词静态向量；BERT 产生双向上下文表示；GPT 自回归预测下一个 token 并擅长生成。
规则: Word2Vec 同一词在不同句中通常同一向量。；BERT 根据左右上下文区分多义词，常用于理解/分类。；embedding 把对象映射到稠密向量，邻近通常代表语义接近。
网站题答案: Q1=BERT
□ BERT/GPT 的上下文向量不是传统词典式固定词向量。

W8.08 图像特征与相似图片搜索

题干: image feature / CNN / ViT / similarity search
结论: 传统特征手工提取边缘/纹理/关键点；深度模型学习 embedding，再用向量近邻检索。
规则: 预处理必须与训练模型要求一致：尺寸、颜色、归一化。；CNN 利用局部卷积；ViT 把图像切成 patches 用 attention。；相似搜索比较特征向量，不是逐像素必须相等。
网站题答案: Q1=向量不在同一可比空间；Q2=图像特征向量
□ 像素距离对平移、缩放和光照敏感，通常不适合作为语义相似度。

W8.09 Metadata：关于数据的数据

题干: metadata / EXIF / GPS / device
结论: Metadata 描述数据对象，可用于过滤、质量验证、连接和解释。
规则: EXIF 可含拍摄时间、GPS、相机型号和方向。；metadata 可结构化存表，但其值仍可能缺失、错误或泄露隐私。；先用工具提取，再统一时区、坐标和单位。
网站题答案: Q1=>；Q2=不同：前者匹配标签，后者匹配输入文件名；Q3=不一定，要检查标签来源、设备时间和其他证据

```
/ : exiftool image.jpg
```

□ 文件修改时间不一定等于拍摄时间；GPS 也不一定存在。

W8.10 高级特征工程

题干: feature engineering / interaction / encoding / selection
结论: 把原始变量转成更能表达问题的特征，并只在训练数据上学习转换。
规则: interaction 捕捉变量组合；polynomial 捕捉非线性；lag/rolling 表达时间依赖。；类别可 one-hot/ordinal/target encode，方法取决于是否有顺序和基数。；selection 降低噪声与成本；AutoML 仍需防泄漏和正确验证。
网站题答案: Q1=Data leakage；Q2=Embedded；Q3=Data leakage
□ rolling/lag 不能使用未来数据；target encoding 必须避免把验证标签泄露进去。

W8.11 Week 8 文件夹 Tutorial：时间序列强化

题干: tutorial / temporal arrays / visual validation
结论: 教程把 W7 的时间序列结构落实为可运行查询：导入、转换、聚合、数组、展开和画图。
规则: 先验证行式结果，再转数组表示。；聚合数组必须显式按 timestamp 排序。；结果数量、起止时间与样例行应在画图前核对。
网站题答案: Q1=Bitcoin 时间序列与 PostgreSQL
□ 图线连续不代表记录顺序正确；绘图工具可能按输入顺序连接。

COMP5339 Quiz 2 完整速答图

W7-W10 每节规则 + 71 道网站自测正确答案 + 代码/陷阱

W9.01 什么是 Data Stream

题干: data stream / unbounded / continuous
结论: Stream
是持续到达、理论上无界的事件序列，不能等全部数据到齐再批处理。
规则: 事件通常有 key、value、timestamp。；核心困难是顺序、迟到、重复、状态和持续输出。；bounded dataset 可完整扫描；unbounded stream 需窗口或增量算法。
网站题答案: Q1=查询持续注册并不断产生新结果
□ streaming 不等于每条都立刻处理；micro-batch 仍是流处理实现。

W9.02 DBMS、DSMS 与两层架构

题干: DBMS vs DSMS / stream architecture
结论: DBMS 对存储数据运行临时查询；DSMS 对持续查询处理不断到来的数据。
规则: DBMS：data persistent, query transient；DSMS：query persistent, data transient。；常见两层：消息传输/broker + stream processing engine。；broker 负责缓冲和分发；处理引擎负责窗口、状态和计算。
网站题答案: Q1=传递、缓冲和路由消息
□ Kafka/MQTT 传消息，不自动完成复杂分析；Spark/Flink 也不等同于消息代理。

W9.03 Publish/Subscribe

题干: publisher / subscriber / broker / topic
结论: 发布者发到 topic，订阅者按 topic 收，broker 解耦双方。
规则: 生产者不需要知道消费者地址或数量。；可扩展多个消费者；保留/重放能力取决于具体 broker。；解耦提高灵活性，但需定义交付语义、顺序和失败处理。
网站题答案: Q1=可以
□ Pub/Sub 不自动保证每个消费者恰好一次收到消息。

W9.04 MQTT：Topic、Callback 与 Wildcard

题干: MQTT / topic / callback / + / #
结论: MQTT 是轻量 Pub/Sub 协议；+ 匹配一层，# 匹配后续所有层且只能在末尾。
规则: client 连接 broker，订阅 topic filter，消息到达触发 callback。；sensor/+/temp 匹配 sensor/a/temp，不匹配 sensor/a/room/temp。；sensor/# 匹配 sensor 下面任意深度。
网站题答案: Q1=home/kitchen/temp

```
/ : client.subscribe('sensor/+/temp')
def on_message(client, userdata, msg): ...
```

□ MQTT topic 是层级字符串，不是文件目录；通配符用于订阅过滤器。

W9.05 MQTT QoS 与端到端语义

题干: MQTT QoS 0 / 1 / 2 / exactly once
结论: QoS0 最多一次；QoS1 至少一次可能重复；QoS2 协议层恰好一次但成本最高。
规则: QoS 只描述客户端与 broker 某一跳的交付。；端到端 exactly-once 还需幂等处理、事务/去重和可靠 sink。；QoS1 常配 event_id 去重，兼顾可靠性与成本。
网站题答案: Q1=Retain flag；Q2=重复交付
□ QoS2 不能单独保证整个业务管道绝无重复副作用。

W9.06 Kafka：Partitioned Durable Log

题干: Kafka topic / partition / offset / consumer group
结论: Kafka 是持久化分区日志；同一 partition 内有序，跨 partition 无全局顺序。
规则: offset 是记录在 partition 内的位置。；同一 consumer group 中一个 partition 同时只分配给一个 consumer。；增加 partition 提高并行度；key 决定分区可保同 key 顺序。
网站题答案: Q1=每个 group 各有实例收到；Q2=否
□ 消费者数超过 partition 数时，多出的消费者会空闲。

W9.07 Stateless 与 Stateful

题干: stateless / stateful
结论: 只依赖当前事件的是 stateless；依赖历史或其他事件的是 stateful。
规则: map/filter 常无状态。；count、dedup、window、running total、join 需要状态。；状态需 checkpoint、过期策略和恢复机制，否则会无限增长或故障丢失。
网站题答案: Q1=Stateful
□ 聚合即使输出一行，也通常是 stateful。

W9.08 Window：把无限流变成有限关系

题干: tumbling / sliding / session / count window
结论: Tumbling 固定不重叠；sliding 固定可重叠；session 按活动间隔；count 按事件个数。
规则: window size 决定统计范围；slide 决定多久输出一次。；sliding 中一条事件可进入多个窗口。；session gap 超过阈值就结束一个会话。
网站题答案: Q1=能，若窗口重叠；Q2=两者都不是；Q3=Count window
□ 5 分钟窗口不说明是否重叠；还要看 slide。

W9.09 Event、Ingestion、Processing Time

题干: event time / ingestion time / processing time
结论: 业务正确性和乱序分析优先 event time；低延迟且顺序不重要时 processing time 更简单。
规则: event time：源头发生；ingestion time：进入平台；processing time：被算子处理。；网络与重试会令到达顺序不同于事件顺序。；processing-time window 结果受机器延迟和负载影响。
网站题答案: Q1=out-of-order events
□ 消息到达时间不是事件发生时间。

W9.10 Watermark 与迟到事件

题干: watermark / late event / allowed lateness
结论: Watermark 是系统对 event-time 进度的估计，用于关闭窗口和清理状态。
规则: 常见概念：watermark ≈ max observed event time - tolerated out-of-orderness。；event_time <= watermark 的新到事件通常视为 late，处理方式取决于配置。；更大等待容忍提高完整性，但增加延迟和状态。
网站题答案: Q1=event-time 进度估计
□ watermark 不是‘此时间之前绝不会再来数据’的绝对保证。

W9.11 Stream–Table 与 Stream–Stream Join

题干: stream-table / stream-stream join
结论: Stream-table 用稳定维表 enrichment；stream-stream 必须按 key 且限定时间范围以控制状态。
规则: 表更新方式决定 enrichment 用最新值还是历史一致值。；双流 join 需要缓存两边尚未匹配事件。；watermark/时间条件用于判断何时可丢弃旧状态。
网站题答案: Q1=限制等待与状态大小

```
/ : ON a.key=b.key AND b.ts BETWEEN a.ts-INTERVAL ... AND
a.ts+INTERVAL ...
```

□ 无时间边界的 stream-stream join 状态可能无限增长。

W9.12 Spark Streaming、Flink 与 MQTT Tutorial

题干: MQTT tutorial / DStream / micro-batch / Flink pipeline
结论: MQTT 送消息，callback 解析消息；Spark 按 micro-batch 计算，Flink 以 continuous pipelining 见长。
规则: 路径：CSV dict→JSON→publisher→broker/topic→subscriber→bytes decode→json.loads→float→window/plot。；先启动 receiver 再启动 publisher；QoS0+retain=False 可能错过早期消息。；DStream 是 RDD 序列；Flink 可把记录立即传给下一个 operator。
网站题答案: Q1=按 topic 接收和转发消息；Q2=还没有注册回调、连接、订阅并运行网络循环；Q3=Spark Streaming micro-batch；Q4=不能，它只验证解析与处理逻辑，没有经过真实 MQTT 网络路径

```
/ : client.on_message=on_message
client.connect(broker,1883)
client.loop_forever()
```

□ Local CSV replay 只验证 callback/plot，不证明真实 MQTT 连接；MQTT 也不是 stream processor。

W10.01 为什么需要可扩展系统

题干: scalability / speedup / scale-up / throughput / latency / elasticity
结论: Scalability 是工作量增大时增加资源仍保持可接受性能；elasticity 是资源随负载自动增减。
规则: Vertical=增强单机；horizontal=增加节点。；Speedup：固定数据与工作，增加资源后完成得更快；理想 speedup S(p)=T1/Tp≈p。；Scale-up：数据量和资源按相同比例增加，响应时间应近似不变。；Throughput=单位时间完成的记录/任务数；latency=response time=完成一个请求所需时间。；评价系统要同时看吞吐、延迟、成本、可靠性，不能只看能否运行。
网站题答案: Q1=Scale-up goal；Q2=Throughput

```
/ : Speedup S(p)=T1/Tp
Scale-up: data×p + resources×p => time=constant
```

□ Speedup 与 scale-up 不是同义词；资源翻倍并不保证性能翻倍，串行部分、shuffle 和 skew 都会限制扩展。

W10.02 Scalable Design Principles

题干: data locality / bottleneck / fault tolerance / observability
结论:
把计算移近数据、消除单点瓶颈、允许失败并可观测，才是可扩展设计。
规则: 并行度受最慢阶段与串行部分限制。；网络 shuffle 常比本地计算昂贵。；重试、复制、checkpoint 提供容错；metrics/logs/traces 定位瓶颈。
网站题答案: Q1=网络搬运
□ 增加机器不能修复倾斜、串行协调器或大量 shuffle。

W10.03 Row、Column、Parquet 与 Wide-column

题干: row / column / Parquet / wide-column
结论: 行存适合整行频繁写与点查；列存适合少数列扫描和聚合；Parquet 是列式文件格式；wide-column 是数据库模型。
规则: 列式可压缩同类型值并跳过无关列。；Parquet 含 row groups、column chunks 和统计信息，利于 predicate pushdown。；wide-column 按 row key/column family 组织稀疏数据，不等于分析型列存。
网站题答案: Q1=Column store
□ columnar storage 与 wide-column database 名字相似但概念不同。

W10.04 Partitioning：把不同子集分到节点

题干: partition / shard key / range / hash / skew
结论: 分区提高并行与容量；好 key 分布均匀且匹配查询，坏 key 造成热点和跨分区扫描。
规则: Range 便于范围查询但可能写热点；hash 均匀但范围查询分散。；co-locate 常连接数据可减少网络。；skew 使少数 partition 过大，整个作业等最慢任务。
网站题答案: Q1=Partitioning
□ partition 越多不一定越快；小文件和调度开销也会增加。

W10.05 Replication：同步、异步与 Leader

题干: replication / leader / synchronous / asynchronous / quorum
结论: 复制提高可用性和读扩展，但引入一致性、延迟和冲突取舍。
规则: 同步复制确认前等待副本：更一致但写延迟高。；异步复制写快，但主节点故障可能丢尚未复制的数据。；leader 处理写并复制；读副本可能读到旧值。
网站题答案: Q1=冲突与收敛；Q2=读到旧副本
□ replication 不是 backup；错误删除也可能迅速复制到所有副本。

W10.06 MongoDB Sharding Architecture

题干: MongoDB sharding / mongos / config server / shard
结论: mongos 路由查询，config servers 保存集群元数据，shards（通常 replica sets）保存数据。
规则: shard key 决定文档分布和查询路由。；查询含 shard key 可 targeted；不含时可能 scatter-gather。；balancer 移动 chunks 保持分布，但迁移有成本。
网站题答案: Q1=路由客户端请求
□ sharding 解决水平容量；replica set 主要解决可用性，两者作用不同。

W10.07 Consistency 与 CAP

题干: CAP / consistency / availability / partition tolerance
结论: 发生网络分区 P 时，只能在一致响应 C 与始终响应 A 之间取舍。
规则:
C：所有节点像单一最新副本；A：每个请求都获非错误响应，但可能旧。；P 是分布式网络必须面对的故障条件，不是平时随意关闭的选项。；CAP 不表示系统永远只能选两个；取舍在分区期间发生。
网站题答案: Q1=最新一致视图
□ CA/CP/AP 标签过度简化；先读题目故障场景和所需保证。

W10.08 HDFS：NameNode、DataNode 与 Read

题干: HDFS / NameNode / DataNode / block / replication
结论: NameNode 保存命名空间和 block 位置元数据；DataNodes 保存实际 blocks；客户端尽量读最近副本。
规则: 大文件切大 block 并复制到多个节点。；客户端先问 NameNode 位置，再直接从 DataNode 读写数据。；HDFS 优化大文件顺序吞吐，不适合大量小文件与低延迟随机更新。
网站题答案: Q1=DataNode
□ NameNode 不承载普通文件数据流；它管理元数据。

W10.09 MapReduce

题干: MapReduce / map / shuffle / reduce
结论: Map 产生 key-value；shuffle/sort 把同 key 数据送到一起；reduce 聚合每个 key。
规则: mapper 可并行处理 partitions。；shuffle 是网络和磁盘重阶段，常是瓶颈。；combiner 可在本地预聚合，但只能用于满足结合/交换性质的操作。
网站题答案: Q1=HDFS上的SQL-like schema/query层；Q2=Shuffle

```
/ : map(record)->(k,v); shuffle group by k;
reduce(k,[v])->result
```

□ shuffle 不是可选排序装饰，而是跨节点按 key 重分布。

W10.10 Spark Architecture 与 Lazy Evaluation

题干: Spark driver / executor / DAG / lazy / transformation / action
结论: Driver 构建 DAG 并调度；executors 处理 partitions；transformation 惰性，action 才触发执行。
规则: map/filter/select 是 transformations；count/collect/write 是 actions。；lineage 支持重算容错；cache/persist 避免重复计算。；宽依赖通常触发 shuffle 和 stage 边界。
网站题答案: Q1=根据lineage重算；Q2=会把结果收回 driver，可能内存不足
□ 调用 transformation 后没有立刻计算；collect 会把全部结果拉到 driver，可能内存溢出。

W10.11 PySpark DataFrame 与 SQL

题干: PySpark DataFrame / Spark SQL / temp view
结论: DataFrame API 与 Spark SQL 使用同一优化引擎；按列操作并尽量在集群端计算。
规则: 常用：select、filter/where、withColumn、groupBy().agg()、join。；createOrReplaceTempView 后可 spark.sql 查询。；显式 schema 比自动推断更可靠；避免 Python UDF 时优先内建函数。
网站题答案: Q1=需要重新注册；Q2=当前 Spark session 中可查询的临时视图

```
/ : df.filter(F.col('x')>0).groupBy('k').agg(F.avg('v'))
df.createOrReplaceTempView('t'); spark.sql('SELECT ... FROM
t')
```

□ 不要在大数据上 toPandas()/collect() 后才处理。

W10.12 Flink 与 Spark

题干: Flink vs Spark
结论: Flink 原生流、低延迟和 event-time/state 强；Spark 擅长统一批流、SQL/ML 生态与 micro-batch。
规则: 两者都可做 batch 与 stream，差异是默认执行模型和生态。；选择依据：延迟 SLA、吞吐、状态规模、团队技能和已有平台。；exactly-once 仍依赖 source、checkpoint 与 sink 共同支持。
网站题答案: Q1=Flink
□ 框架名字本身不能保证端到端语义或性能。

W10.13 Week 10 Tutorial：本地 Spark 到真实结果

题干: local Spark tutorial / Bitcoin / real result
结论: 本地 single JVM 用相同 DataFrame/DAG 模型验证逻辑，但不能证明集群可扩展性。
规则: 先 schema/sample/count，再转换、聚合并用 explain 看计划。；通过分区数、shuffle、skew 和 collect 风险判断扩展性。；最终结果必须可由代码重现并附输入、过滤边界和统计核对。
网站题答案: Q1=减少送回 driver 的数据量
□ local[*] 并不代表真正多节点；本地成功不能证明生产性能。